



Tecnológico de Monterrey

INGENIERÍA EN ROBÓTICA Y SISTEMAS DIGITALES

ESCUELA DE INGENIERÍA Y CIENCIAS

TE2003B – Diseño de Sistemas en Chip

Pipeline

Gerardo Maximiliano López Herrera Matrícula: A01614489

Bryan Zahir Flores Pérez Matrícula: A01742293

Alejandro Xie Hu Matrícula: A01743620

1. Breve descripción del funcionamiento del procesador

El proyecto consiste en el diseño e implementación en Verilog de un procesador con pipeline. A diferencia de un procesador singlecycle que ejecuta una instrucción completa por ciclo de reloj, este diseño divide el procesamiento en múltiples etapas simultáneas, permitiendo que varias instrucciones se encuentren en diferentes fases de ejecución al mismo tiempo. Esto incrementa significativamente el throughput del sistema. El diseño opera con una señal de reloj de 100 MHz (periodo de 10 ns) y cuenta con un testbench que valida su funcionamiento completo.

2. Explicación del pipeline de 5 etapas

La arquitectura del procesador está dividida en las 5 etapas:

1. **Instruction Fetch (IF):** Se utiliza el Program Counter (PC) para buscar la siguiente instrucción en la Memoria de Instrucciones (ROM).
2. **Instruction Decode (ID):** Se decodifica la instrucción obtenida, se leen los operandos desde el Banco de Registros y se extienden de signo los valores inmediatos.
3. **Execute (EX):** La ALU realiza las operaciones matemáticas o lógicas necesarias, o calcula las direcciones de memoria.
4. **Memory Access (MEM):** Si la instrucción es un Load o Store, se accede a la Memoria de Datos (RAM) para leer o escribir información.
5. **Write Back (WB):** El resultado de la ALU o el dato leído de la memoria se escribe de regreso en el Banco de Registros para que pueda ser usado por instrucciones futuras.

3. Funcionamiento de la unidad de hazard

La segmentación introduce dependencias de datos y de control conocidas como *Hazards*. Para solucionarlos, se implementó una **Hazard Unit** (Unidad de Control de Riesgos) que opera mediante dos mecanismos principales:

- **Forwarding:** Cuando una instrucción en la etapa EX necesita un dato que acaba de ser calculado por una instrucción anterior en las etapas MEM o WB (pero que aún no se escribe en los registros), la Hazard Unit detecta la dependencia mediante la comparación de las direcciones de los registros (**rs**, **rt**, **rd**) y redirige el dato directamente a las entradas de la ALU usando multiplexores (**forward_a** y **forward_b**).
- **Stalls:** En el caso de un *Load-Use Hazard* (cuando una instrucción necesita un dato de un *Load* que apenas se está leyendo de la memoria), el *forwarding* no es suficiente. La unidad detecta esto (**ex_mem_read**) y emite una señal de **stall** que detiene el PC y la etapa IF por un ciclo, insertando una burbuja en el pipeline para dar tiempo a que el dato esté listo.

4. Capturas de simulación

A continuación, se presenta la evidencia de la simulación del módulo *Top-Level (DUT)* ejecutado en el entorno de ModelSim:

